

Drzewo

nieliniowa struktura do przechowywania danych, przydatna do przechowywania danych uporządkowanych **hierarchicznie**, np. strukturę folderów i podfolderów na dysku. Do opisu drzewa potrzebujemy jakoś zdefiniować abstrakcyjną złożoną strukturę - już nie konkretny typ, przechowujący dany rodzaj liczby, czy tablicę liczb.

Matematycznie, drzewo to przykład grafu spójnego i acyklicznego. Graf to zbiór węzłów/wierzchołków oraz krawędzi/połączeń między nimi (jedna krawędź łączy dwa wierzchołki). **W drzewie nie może być cykli** i wszystkie wierzchołki muszą mieć jakieś połączenie z resztą grafu.

Przykłady grafów

1.

Wierzchołki: u_1, u_2, u_3, u_4 :

```
u1 - u2
  \ /
   u3
   |
   u4
```

Krawędzie: $\{u_1, u_3\}; \{u_2, u_3\}; \{u_3, u_4\}; \{u_1, u_2\}$. Istnieje cykl: $u_1 - u_2 - u_3 - u_1$, czyli ścieżka, po której mogę przejść z u_1 z powrotem do u_1 bez konieczności powracania. Zatem to nie jest drzewo.

2.

Wierzchołki u_1, u_2, u_3, u_4 :

2.1:

```
    u1
   /  \
  u2   u3
 /
u4
```

krawędzie: $\{u_1, u_2\}; \{u_2, u_4\}; \{u_1, u_3\}$ OK, nie ma cykli, to drzewo.

2.2:

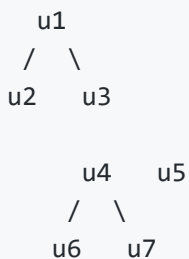
u1

2.3:

u1 - u2

Powyższe to też drzewa. 2.2: nie ma cykli, tylko jeden wierzchołek, zatem wszystkie wierzchołki są połączone (tzw. *pusta prawda*: założmy, że nie, tj. mamy jakiś cykl. To by oznaczało że mamy jakąś ścieżkę po której możemy przejść z u1 do u1 bez konieczności zawracania. Ale to niemożliwe, bo mamy tylko jeden węzeł). . 2.3: dwa wierzchołki, nie rozgałęzień, ale nie ma też cykli. OK).

3.



Mamy 7 wierzchołków. Nie ma cykli (to dobrze), ale brakuje nam krawędzi-- mamy trzy odrębne komponenty grafu, pomiędzy którymi nie ma połączeń: {u1,u2,u3}; {u6,u4,u7}; {u5} . To nie jest drzewo.

Struktury typów złożonych

Analogicznie do `typedef struct { } my_name` w C (lub szablonów w `c++`) będziemy je definiować pisząc:

`NAZWA {pole1, pole2 ... } : typ_pole1, typ_pole2...` . Przykłady:

Struktura przechowująca współrzędne punktu w przestrzeni $\mathbb{R}^2$ (na płaszczyźnie):

- `POINT {x,y}`: $\mathbb{R}, \mathbb{R}$. jedna instancja typu `POINT` przechowuje `x,y` - oba z nich to liczby rzeczywiste.
- Deklarujemy trzy punkty `x1, x2,x3` i ustawiamy im jakieś wartości (dostęp do pól przez `.` jak w `C`):

- Niech X_1, X_2, X_3 to POINT
- $X_1.x = 0; X_1.y = 1$
- $X_2.x = 1; X_2.y = \pi$
- $X_3.x = -3; X_3.y = 2.7$
- Wtedy `x1;x2;x3` przechowują dane odpowiednio 3 punktów: $(0, 1); (1, \pi); (-3; 2.7)$

Struktura węzła `Node` - budulec drzewa

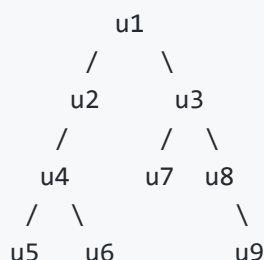
Jeśli mamy drzewo **binarne** (tylko takie nam będzie potrzebne) i chcemy przechowywać w drzewie liczby rzeczywiste, moglibyśmy zdefiniować następującą strukturę:

`NODE{label, v, child_left, child_right} :  $\mathbb{N}, \mathbb{R}, \mathbb{N}_0, \mathbb{N}_0$`

gdzie:

- `label` - etykieta danego wierzchołka/węzła `NODE` -a. Wybieramy tutaj akurat liczby naturalne do ich numerowania. **Etykiety nie mogą się powtarzać- nie mogą istnieć dwa różne węzły z tą samą etykietą.** `label` NIE JEST wartością przechowywaną w danym węźle.
- `v` -przechowywana wartość w węźle - napisaliśmy, że chcemy przechowywać liczby rzeczywiste, zatem tu mamy typ $\mathbb{R}$.
- `child_left` , `child_right` - etykiety "dzieci" węzła. Każdy węzeł może mieć 2,1 lub zero dzieci (w drzewie *binarnym*). Jeśli węzeł nie ma dziecka "po lewej" , wtedy `child_left=0` . Gdy nie ma dziecka "po prawej", wtedy `child_right=0` . Jak nie ma obu, wtedy oba pola są zerami (**węzeł jest liściem**). Właśnie dlatego dla pola `label` mamy typ $\mathbb{N}$ (liczby naturalne BEZ ZERA) a w `child_left` i `child_right` mamy $\mathbb{N}_0$ (liczby naturalne z zerem), aby móc zakodować brak dziecka (stawiając wartość zero).

Przykładowe drzewo



Np. powyżej mamy 9 węzłów, zatem potrzebowalibyśmy 9 struktur `NODE`, aby to opisać w kodzie. Np. jeśli `u2.label=2` oraz `u3.label=3`, to dla `u1` mamy ustawione: `u1.child_left=2;` `u1.child_right=3`.

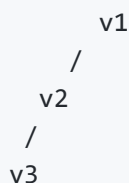
Z kolei jeśli `u9.label=9`, to: `u8.child_right=9` a `u8.child_left=0` (bo brak dziecka "z lewej strony"). Z kolei `u9` nie ma w ogóle dzieci, więc `u9.child_left=0;` `u9.child_right=0` (jest liściem).

Implementacja w C

...Zazwyczaj nie używa się `int`-ów do numerowania wierzchołków i określania kto jest synem kogo, tylko **wskaźników**. `child_left` byłby wskaźnikiem na strukturę `NODE` "lewego syna", a `child_right` wskaźnikiem na prawego. Nie potrzebujemy też osobnego pola `label`, bo każdy węzeł jest identyfikowany przez swój adres. Ale szczerze - drzewo to dość intuicyjny twór, którego bardzo łatwo pograć w detale implementacji przez wskaźniki w C. A to nie jest istota tego tworu.

Lista połączona jednokierunkowa jako rodzaj drzewa

Można rozumieć jako przypadek drzewa, gdzie każdy węzeł ma co najwyżej jedno dziecko:



Zakładając `v1.label=1`, `v2.label=2`, `v3.label=3`, `v1.left_child=2`, `v2.left_child=3`, `v3.left_child=0`. A wszystkie pola `right_child` wszystkich wierzchołków są zerami.

Wniosek: implementując listę, nie potrzebujemy dwóch etykiet "dzieci"- wystarczy jedna.

W liście jednokierunkowej wyróżniamy raczej węzły `head` oraz `tail` zamiast korzenia i liści. Także **korzeń** takiego zdegenerowanego drzewa to będzie `head` odpowiadającej listy, a pojedynczy liść `v3` - `tail`:

```
(head) v1 -> v2 -> v3 (tail)
```

Skierowane a nieskierowane drzewa

Na powyższym diagramie połączenia są oznaczone strzałkami. Oznacza to, że w tym grafie `(v1,v2)` należy do zbioru krawędzi, a `(v2,v1)` już nie (bo `v1 -> v2` ale nie ma łączenia `v2 ->`

v1). Także jest grafem skierowanym.

Czy drzewa musimy rozumieć jako grafy skierowane?

ODP: NIE, jeśli wyraźnie określimy, który węzeł jest korzeniem. Np. dla podanego grafu nieskierowanego:

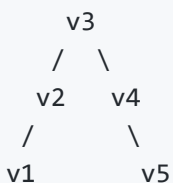
v1 - v2 - v3 - v4 - v5

Mogę narysować wiele drzew, które będą miały takie same zbiory wierzchołków i krawędzi:

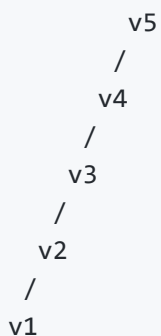
#1



#2



#3



Ale jak już określić, który wierzchołek jest **korzeniem**, nie ma już sytuacji niejednoznacznej. Dla narysowanych 3 grafów u góry korzeniami będą odpowiednio v2 , v3 , v5 , a reszta wynika z tego, kto jest z kim połączony.

Natomiast dodając kierunki łączeniom, co jest korzeniem mozna wywnioskować z samych połączeń -- do korzenia nie wchodzą żadne strzałki (tylko z niego wychodzą):

```
#1 v1 <- v2 -> v3 -> v4
#2 v1 <- v2 <- v3 -> v4 -> v5
#3 v1 <- v2 <- v3 <- v4 <- v5
```

Szukanie danych w drzewie

Jeśli drzewo to stuktura danych - w każdym wierzchołku (strukturze `NODE`) przechowujemy też jakąś wartość (w polu `v`). **Motywacja:** Nie wszystkie dane jest sens trzymać w tablicy -- np. struktura folderów dobrze pasuje do drzewa. Przykładowo, jeśli mam taki folder:

```
games\
  Tibia\
    Tibia.exe
    user_password.txt
  Super Silent Hill Mario Blaster 3D\
    SSHMB3D.exe
    savefiles\
      save1.txt
      save2.txt
```

Mielibyśmy wierzchołek na każdy folder i plik, czyli w sumie 9 wierzchołków. Jako pole `.v` każdego węzła (przechowujące wartości) moglibyśmy wstawić nazwę folderu lub pliku (*to duużę uproszczenie rzeczywistości*).

Diagram, na którym zamiast pisać oznaczenie wierzchołka, wstawiamy jego wartość:

```
      games
     /    \
  Tibia    Super Silent Hill Mario Blaster 3D
   /  \           |           \
Tibia.exe user_password.txt SSHMB3D.exe  savefiles
                                   /      \
                               save1.txt save2.txt
```

Znalezienie danej wartości w drzewie polega na:

- sprawdzeniu wierzchołków, aby określić, czy wartość jakiegokolwiek z nich odpowiada temu, czego szukamy.
- szukając danej wartości-- musimy posłużyć się zdefiniowanymi strukturami `NODE` oraz powiązaniami, jakie definiują, tj. w jednym kroku, mogę przejść z `v1` do `v2` , tylko gdy

węzeł `v2` jest dzieckiem `v1`.

DFS (Depth First Search), szukanie w głąb

Polega na przeszukaniu w pierwszej kolejności każdej z gałęzi w głąb (od korzenia do liścia). Zaczynamy zawsze od korzenia i idziemy według poniższych kroków. Mamy różne warianty, które różnicują kolejność odwiedzania: rodzic, lewe dziecko i prawe dziecko, najpopularniejsze to: `preorder`, `postorder`, `inorder`. My skupimy się na tylko jednym:

```
DFS_preorder(x typu NODE, szukana_wartosc):
```

1. Wypisz `x.v`.
2. Zwróć `TRUE` jeśli `x.v` to szukana wartość.
3. Jeśli `x.left_child != 0`: uruchom `DFS_preorder(y)`, gdzie `y` to węzeł o etykiecie równej `x.left_child` (przeszukaj poddrzewo po lewej od `x`), no i przypisz wynik do zmiennej `L := DFS_preorder(y)`. Jeśli `L` to `TRUE`, zwróć `TRUE`.
4. Jeśli `x.right_child != 0`: uruchom `DFS_preorder(y)`, gdzie `y` to węzeł o etykiecie równej `x.right_child` (przeszukaj poddrzewo po prawej od `x`), przypisz wynik do zmiennej `R := DFS_preorder(y)`. Jeśli `R` to `TRUE`, zwróć `TRUE`.
5. Jak tu dotarłeś i nic nie zwróciłeś -- zwróć `FALSE`.

Jest to formuła rekurencyjna. Zatem jeśli `u1` to korzeń, zaczynamy od wywołania `DFS_preorder(u1, moja_szukana)` i pozwalamy rekurencji szukać `moja_szukana`. Jeśli `moja_szukana` jest gdzieś w drzewie - otrzymamy `TRUE`, inaczej `FALSE`.

***uwaga odnośnie pkt 1**: Ta instrukcja nie jest konieczna. Dodajemy ją po to, aby śledzić kolejność "odwiedzania" wierzchołków w drzewie.

Przykład zastosowania dla drzewa z poprzedniego przykładu: kolejność wypisywanych wartości w węzłach odpowiada kolejności odwiedzania ich w drzewie. Załóżmy, że zmienna `korzen` typu `NODE` trzyma dane korzenie drzewa, tj. `korzen.v = "games"`, a jego "lewe" i "prawe" dzieci to odpowiednio węzły trzymające wartości: `"Tibia"`, `"Super Silent Hill Mario Blaster 3D"`

```
DFS_preorder(korzen, "save1.txt") //zobaczmy:

"games"
"Tibia"
"Tibia.exe"
"user_password.txt"
"Super Silent Hill Mario Blaster 3D\"
"SSHMB3D.exe"
"savefiles"
"save1.txt" // koniec, znaleźliśmy.
//nie uruchomi się szukanie prawej gałęzi z "savefiles\"
```

Inny przykład: szukamy wartości "solitaire" (nie znajdziemy tego):

```
DFS_preorder(korzen, "solitaire")
"games"
"Tibia"
"Tibia.exe"
"user_password.txt"
"Super Silent Hill Mario Blaster 3D"
"SSHMB3D.exe"
"savefiles"
"save1.txt"
"save2.txt"
//koniec, nie ma nic więcej
```

Tutaj z kolei kolejny przykład pozytywny:

```
DFS_preorder(korzen, "SSHMB3D.exe")
"games"
"Tibia"
"Tibia.exe"
"user_password.txt"
"Super Silent Hill Mario Blaster 3D"
"SSHMB3D.exe" // jest, koniec.
```

Zadanie

Celem zadania jest **poprawne zrozumienie kolejności odwiedzania węzłów w DFS "preorder" oraz czym jest drzewo.**

1. Przepisz na kartkę przydzielone Tobie drzewo w orientacji pionowej z listy poniżej (korzeń u góry, liście na dole).
2. Wypisz: jaką wartość przechowuje korzeń, a jaką liście.

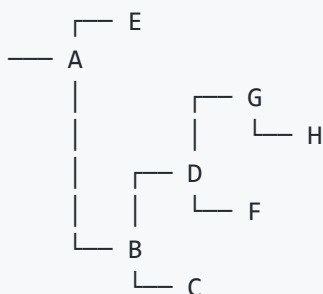
3. Potem, pokaż, w jakiej kolejności `DFS_preorder` odwiedza węzły w tym drzewie, tak jak w powyższych przykładach:

1. Pokaż jeden przykład **pozytywny** - wyszukanie wartości w liściu, która faktycznie jest przechowywana
2. i drugi przykład **negatywny** - wartości, która nie jest przechowywana w żadnym węźle drzewa (czyli `DFS` przechodzi przez całe drzewo i nic nie znajduje).

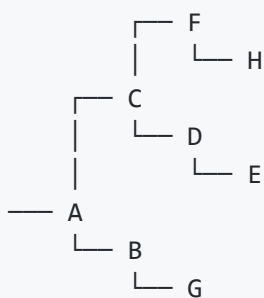
Drzewa (sprawdzić, które mamy przydzielone w mailu):

dół/góra odpowiadają kierunkom lewo/prawo na poprzednich diagramach. W miejscu każdego węzła umieszczamy tutaj **wartość**, jaką **przechowuje**.

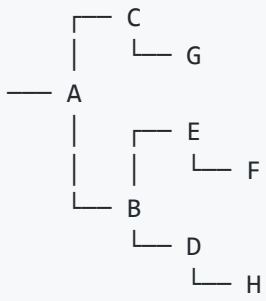
1.



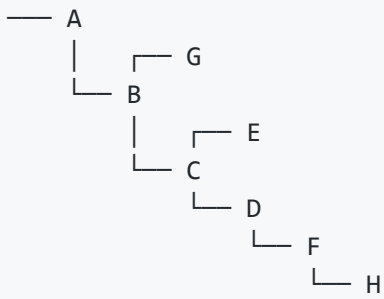
2.



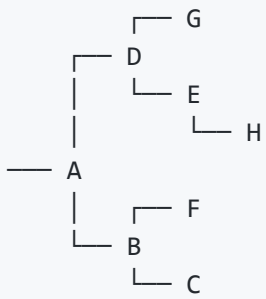
3.



4.



5.



6.

